

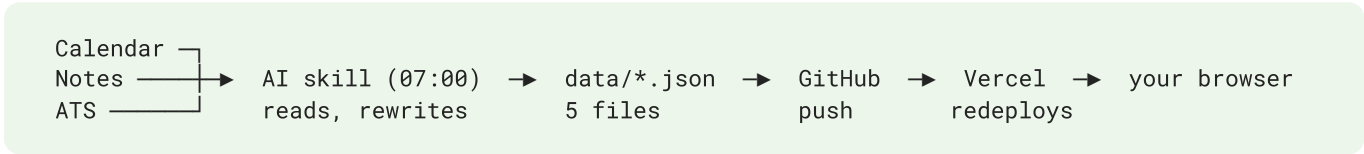
The dashboard with no database

A management dashboard that updates itself every morning. GitHub, Vercel and one AI skill. Nothing else. · David Dwyer-Whitmore, Scede

What we are building

A one-page dashboard, live on the web, for a talent leader: today’s meetings with prep, the to-do list pulled from meeting notes, and what the team and open roles are doing. It refreshes before you open your laptop.

The trick that keeps it simple: there is no database. The data is five small text files inside the website’s code. An AI skill rewrites them each morning and pushes to GitHub. Vercel notices and republishes the page. That is the whole system. Not the best architecture. The easiest one to understand completely and get working in two hours.



You need

1. **GitHub account** (github.com, free). Where the code lives.
2. **Vercel account** (vercel.com, sign in with GitHub, free). Turns the code into a website.
3. **An AI tool that can write code.** Claude or ChatGPT is the smooth path: connected to GitHub, the skill pushes for you; not connected, it hands you the files and you paste them.

Gemini: build the page in Gemini Canvas, export the code, upload it. The morning skill is a Gem that hands you the files.

Copilot: Shay’s session today; this one at home with Copilot Studio.

The five files

FILE	HOLDS
meta.json	As-of date, the five headline numbers
today.json	Top three, today’s meetings, prep bullets
actions.json	To-dos with owner, due, priority, source
team.json	Hires vs target, weekly screens per recruiter
roles.json	Open roles, stage counts, health flag

SCHEMA.md is the contract: field names, allowed values, rules. Your AI writes it, then reads it before every update.

The five steps · about 60 minutes

STEP	YOU SAY	YOU GET	
0	Accounts, then connections · 5 min	GitHub, Vercel, connect your AI to GitHub if it can	Ready to build
1	Build the page · 15 min	Build me a one-page dashboard as a static site that reads JSON files from /data ...	index.html, styles.css, app.js, data/

2	Put it on GitHub · 5 min	Push it to a new GitHub repo or upload at github.com/new	A repository
3	Put it on the web · 5 min	Vercel: Add New Project, import, leave defaults, Deploy	A live URL
4	Write the skill · 20 min	Write a skill that rewrites the data files from my calendar, notes and ATS export, then commits and pushes	SKILL.md or Custom GPT instructions
5	Run it, then schedule it · 10 min	Run it for today then Scheduled tasks, weekdays 07:00	It runs without you

Full prompts overleaf. Read what comes back before you send the next one. Ask for changes in plain English: *move the actions under the team table*. That is vibe coding.

THE PROMPTS, IN ORDER · SWAP THE BRACKETED PARTS FOR YOUR OWN

1 · BUILD THE PAGE

Build me a one-page personal dashboard as a static website: one index.html, one styles.css, one app.js, no framework, no build step, so it can be deployed on Vercel as-is. The page reads its data from JSON files in a data/ folder using fetch, and renders: (1) a greeting with today's date, (2) five headline numbers, (3) today's meetings with prep notes under each, (4) an actions list with owner, due date and priority, (5) a table of my team with this week's activity, (6) a table of open roles with stage counts and a health flag. Create realistic example data for a talent team at a [200-person tech company]. Use the font Manrope from Google Fonts, a light background, white cards, one accent colour [#1AA56A]. Make it work on a phone. Write a data/SCHEMA.md that documents every field so a future automation can rewrite the files safely.

2 · PUT IT ON GITHUB

Create a new private GitHub repository called [my-dashboard], add all the files, and push to the main branch. Tell me the repository URL.

Not connected? github.com/new → "uploading an existing file" → drag the files in → commit.

3 · DEPLOY IT (NO PROMPT)

vercel.com → Add New Project → import the repository → leave every setting as default → Deploy. Ninety seconds later you have a URL. Every push to main redeploys it.

4 · MAKE THE DATA UPDATE ITSELF

Write a skill called update-dashboard (a SKILL.md file in .claude/skills/update-dashboard/) that, when run, reads today's calendar, my meeting notes from the last 24 hours and the latest ATS export I give it, rewrites the five JSON files in data/ following data/SCHEMA.md, validates them, commits only the data files with the message chore(data): daily update YYYY-MM-DD and pushes to main. It must keep ids stable, never invent a number, and never include a candidate's name. Reply in five lines.

ChatGPT: ask for the same thing "as instructions for a Custom GPT", paste into the Instructions box, attach SCHEMA.md. Gemini: the same as a Gem; it hands you the files to paste.

5 · RUN IT ONCE, THEN SCHEDULE IT

Run the update-dashboard skill for today. Here is my ATS export: [paste or attach].

Then: Claude → Scheduled tasks → "Run the update-dashboard skill", weekdays 07:00. ChatGPT → Tasks, same wording. The skill you have to remember to run is the skill you will stop running.

Four rules worth stealing

Never invent a number.

Source missing? Keep yesterday's value and say so.

No candidate names. Ever.

Counts only. The ATS is the system of record for people. GDPR, and what lets you share the URL.

Keep ids stable.

Action a-101 today is a-101 tomorrow. It is how the page tracks what got done.

The skill only touches data/.

The page code changes when you ask, in a conversation. The morning run never edits it.

If your laptop is locked down

Take it further, in this order

- **AI can't connect to GitHub:** ask for the files, paste them into GitHub's web editor.
- **AI can't connect to your ATS:** export a report each morning and attach it. Automate the export later.
- **AI can't see your calendar:** paste today's agenda.
- **Can't create a GitHub account:** not today. Shay's session needs nothing external.

Every workaround is the same shape: the AI does the thinking, you do the copy and paste. Slower, not worse.

1. **Add a section:** a "waiting on hiring manager" list counting candidates stuck more than 3 days.
2. **Connect the source:** replace the export with a connector so the skill reads the ATS or calendar directly.
3. **Put it behind a login** before real data goes in: your SSO, or Vercel deployment protection.
4. **Then a database.** When more than one person needs to write, move data/ into Supabase (free tier). That is where the shortcut stops paying for itself.

Example repository github.com/daviddw-scede/bbb-management-dashboard

Live example bbb-management-dashboard-daviddw-6234s-projects.vercel.app

Show and tell 14:00. Bring your URL, not your slides.

QR to the live
example goes
here